DEPLOYMENT · FIELD NOTES

VidLingo 部署手记

一个晚上把项目送上线的完整拆解：三层拓扑、Docker 在哪一环、CI/CD 其实已经在跑，以及一套换个项目还能用的心智模型。

项目	产物	计算
vidlingo	Docker · 359 MB	Render
状态	**日期**	
Neon PG 18.6	2026-08-20	

01 三层：计算 / 状态 / 产物

02 Docker 在哪一环

03 CI/CD 已经在跑了

04 五个问题

05 面试问答

06 今晚踩的四个坑

01

三层：计算 / 状态 / 产物

部署的本质是给三样东西找地方待着。名词很多，但结构只有三层。

要放的东西	是什么	我们用了	别的选择
计算	跑你的进程的 CPU 和内存	**Render**	AWS ECS / Fly.io / 自己的 VPS
状态	重启后必须还在的数据	**Neon**	AWS RDS / Render 自带 Postgres

产物	要跑起来的那个东西	**Docker 镜像**	jar 包 / 一个 git 仓库

Neon 是托管 Postgres。就是一台替你运维的数据库服务器——不用自己装、不用配备份、不用管磁盘满了。它的特别之处是**存储和计算分离**：没请求时计算部分缩到零，所以免费额度才给得起，代价是冷启动那半秒。

Render 是 PaaS（平台即服务）。你给它一个 Dockerfile，它负责构建、运行、发 HTTPS 域名、进程挂了自动重启。

Blueprint 不是服务器

它是 Render 对 **Infrastructure as Code** 的叫法，实体就是仓库里那个 `render.yaml`：

RENDER.YAML · 节选

```
services:
  - type: web
    name: vidlingo
    runtime: docker
    healthCheckPath: /api/health
    envVars:
      - key: DATABASE_URL
        sync: false        # 值不在这个文件里，手填在面板上
```

不用它也能部署——在网页上一个个点选项就行。但那样"这个服务是怎么配的"只存在于那个网页和你的记忆里。**用了 Blueprint，基础设施配置进了 git**：任何人 clone 仓库就能看到完整拓扑，改配置要走 code review，回滚就是 revert。



一个容器同时是前端和后端。客户端和 API 在同一个 origin，所以浏览器根本不会发起跨域请求——**CORS 这个问题被架构消掉了，而不是被配置解决了**。容器本身是一次性的：状态一律外置到 Neon。

为什么不用 AWS

AWS 完全能做。而**"为什么没用"才是面试时更该讲清楚的东西**：

> **可以直接这么答**
>
> "我用的是 Render + Neon，但架构上是 provider-agnostic 的。产物是一个 Docker 镜像，配置全走环境变量，换成 ECS Fargate + RDS 就是同一个镜像换个地方跑。选 Render 是因为项目要在一个晚上上线，AWS 那套 VPC、安全组、IAM、ALB 的配置对这个规模是过度投入。"

	RENDER + NEON	AWS
首次上线耗时	约 10 分钟	半天到两天
要配的东西	5 个环境变量	VPC、子网、安全组、IAM、ECR、ECS 任务定义、ALB、RDS 参数组
花钱	免费层，不绑卡	12 个月免费层，**要绑卡，容易误超扣钱**
简历说服力	够用，前提是讲得清取舍	更高，前提是你真的懂那套

如果投的岗位 JD 里明写了 AWS，那值得再花一天搬到 ECS 上——**镜像已经有了，搬迁的工作量在 AWS 那边的配置，不在你的代码里**，这本身就证明了架构是解耦的。

为什么数据库要重置密码

> **凭据卫生**
>
> 不是因为泄露了，是因为它**离开了它该待的地方**。那个密码本来只该存在于三处：Neon 的数据库、本机 gitignored 的 `.env`、Render 的环境变量面板。一旦贴进聊天框，它就还存在于对话记录文件和别人的服务器上——这些地方不公开，但**不在你的控制范围内，也无法审计谁读过**。
>
> 安全上的标准做法：凭据一旦离开预定通道，就当它已经泄露处理，**因为你无法证明它没有**。重置成本是十秒，不重置的代价是你永远不确定。

02

Docker 在哪一环

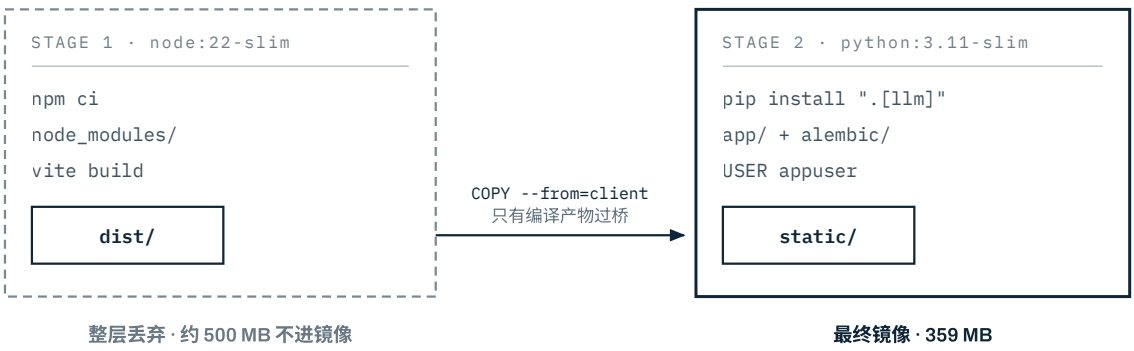
你确实用到了，而且它是整条链路的核心。先分清三个词——面试常问。

词	比喻	在这个项目里
Dockerfile	菜谱	仓库根目录那个文本文件
Image 镜像	按菜谱做好的冷冻餐	`vidlingo:local`，359 MB，不可变、可分发
Container 容器	把冷冻餐加热吃	镜像的一个运行实例，`docker run` 起来的那个

你今晚亲手走完的循环

```
docker build -t vidlingo:local .          # 菜谱 → 镜像
docker run -d -p 8080:8000 \              # 镜像 → 容器
  --env-file neon.env vidlingo:local
curl -s localhost:8080/api/health         # 验证 → 发现 404 那个 bug
#    ↑ 改代码，然后从第一行重来
```

多阶段构建：为什么是 359 MB 而不是 1 GB



Node 工具链只是脚手架，不是产品。第一阶段几百兆的 `node_modules` 在构建结束后整个消失，只有几百 KB 的编译产物被复制进第二阶段。**虚线框表示这一层不会出现在最终镜像里。**

真正的价值不在体积，而在：**这个镜像在你的 Windows 上、在 GitHub 的 Ubuntu**

runner 上、在 Render 的服务器上，行为完全一致。"在我机器上是好的"这个问题被从根上消掉了。

而这不是理论——今晚它**两次**抓到了只在干净环境暴露的 bug（见 06）。

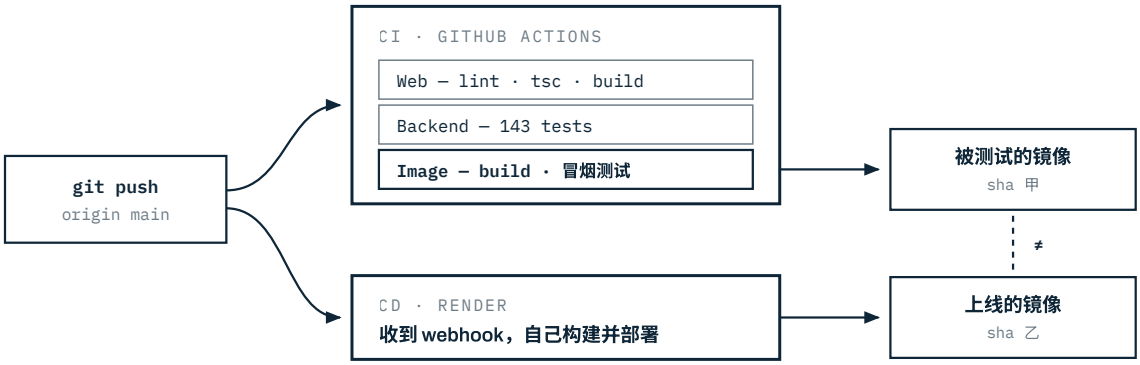
"本地部署"这个说法要拆掉

> **概念纠正**
>
> 在本机 `docker run` **不是部署，是验证**——关掉电脑它就没了，外网也访问不到。等 Render 跑起来、拿到公网 URL，那才叫部署。两者跑的是同一个镜像，这正是本地验证有意义的原因。

03

CI/CD 已经在跑了

顺序不是"先本地部署，以后再上 CI/CD"。CI 从你第一次 push 就在跑，CD 在你点下 Apply 的那一刻生效。



当前方案的边界，就在最右边那条虚线上。Actions 和 Render 各自 build 一遍，所以**测过的镜像和上线的镜像不是同一个**。更成熟的做法是 Actions 构建完推到 registry（GHCR / ECR），Render 直接拉现成的镜像。**面试时主动提这一点很加分**——它显示你知道自己方案的边界在哪。

另外，容器的 `ENTRYPOINT` 在启动服务之前会先跑一次 `alembic upgrade head`。**所以数据库迁移也是自动的**：你改了表结构，push 一下，线上的库会在新容器起来时自己升级。

04

五个问题

"换个项目让我从头搞还是不会"——因为记的是步骤，不是结构。步骤换个项目就失效，结构不会。**任何部署本质上都在回答这五个问题。**

01 **代码怎么变成一个能跑的产物？**

Dockerfile 多阶段构建 → 一个 359 MB 的镜像

02 **这个产物在哪台机器上跑？**

Render 的 Docker runtime，免费实例

03 **重启后必须还在的数据放哪？**

Neon Postgres。**绝不放在容器里**——容器的文件系统是一次性的

04 **配置和密钥怎么进去？**

环境变量。`sync: false`，只在面板上填，永不进 git

05 **外部流量怎么进来？**

Render 的 HTTPS 域名 → 容器的 `$PORT`

再加两个成熟度问题，答不上来说明方案还没做完：

- **怎么知道它活着？** `healthCheckPath: /api/health`。Render 靠它判断部署是否成功，失败就不切流量。
- **挂了怎么退回去？** Render 保留历史版本，一键 rollback。配置的回滚则是 `git revert`。

> 最常踩的一条
>
> 第 03 条。新手常把 SQLite 文件放在容器里，然后每次重新部署数据全没了——**因为容器的文件系统随容器销毁**。这个项目从一开始就把状态外置到 Postgres，不是巧合，是这一条的直接结果。

下次换个项目，就按这五格逐个填空。语言、框架、云厂商全变了，这五格不变。

05

面试问答

Q 讲讲你这个项目是怎么部署的。

A 一个 Docker 镜像，多阶段构建——前端在 Node 阶段编译，产物拷进 Python 运行时阶段，Node 那层丢掉。前后端是同一个容器，所以不存在 CORS，客户端和 API 同源。跑在 Render 上，数据库是 Neon 的托管 Postgres。基础设施用 `render.yaml` 声明，进 git；密钥走 `sync: false`，只在面板上填。

Q 为什么前后端不分开部署？

A 分开要处理 CORS、两套部署流程、两个域名。合成一个容器之后，开发和生产的拓扑完全一致——Vite 在本地把 `/api` 代理到后端，容器里 FastAPI 直接同时服务静态文件和 API，两边路径一模一样。代价是前端没走 CDN，这个量级不值得为此付复杂度。

Q 数据库为什么不用平台自带的？

A Render 的免费 Postgres 30 天后会被删。这个链接要放在简历上，不能有这种失效模式。Neon 免费层持久，而且换 provider 只是改一个环境变量——连接串的驱动前缀 `驱动前缀` 我在配置层统一改写了，从任何面板复制粘贴都能连。

Q CI 里跑什么？

A 三个 job。前端 lint + 类型检查 + 构建；后端 lint + 143 个测试；第三个 job 真的 `docker build`，把容器跑起来，打健康检查和深链接。第三个最有价值——它抓到过两个只在干净环境出现的 bug。

Q 密钥怎么管的？

A 不进 git。`.env` 在 gitignore 里，仓库里只有 `.env.example` 的空占位。生产上填在面板，`render.yaml` 里标 `sync: false`。每次 push 前扫一遍 outgoing diff。

Q 如果 key 不小心泄露了怎么办?

A 第一步 **吊销和** ，不是删 commit。git 历史可以改写，但 key 已经暴露过了，改写永远是 **轮换** 历史不能让它变回安全。清理历史是第二步。

Q 这套方案有什么你不满意的地方?

A CI 和 Render 各自 build 一遍镜像，所以测过的和上线的严格来说不是同一个 artifact。要修就是 Actions 推 GHCR，Render 拉现成镜像。另外免费实例闲置会休眠，冷启动约一分钟——对简历链接可以接受，对真实用户不行。

练习建议

今晚这一遍走完之后，找个周末**拿一个 Flask hello world 从零再搭一次**，不看命令，只按 五个问题 逐格填空。第二次会快得多，而且那时候才算真的会。今晚这一遍的价值在于——**你现在知道坑长什么样**，这是看教程学不到的。

06

今晚踩的四个坑

这一节是整份文档里**面试价值最高**的部分。"我配了 CI" 是流程描述，"CI 抓到了什么、为什么只有它能抓到" 才是经验。

CI 抓到 **API 把前端页面盖住了**

`/vocab` 既是客户端页面又是 API 接口。两边都挂在根路径上时 API 赢了，浏览器访问 `/vocab` 拿到的是 `401 Not authenticated`。**修法不是加特例，是把整个 API 挪到 `/api` 前缀下**——让两个命名空间从结构上不可能再撞。

CI 抓到 **测试偷用了我本机的开发数据库**

一个 fixture 自己调 `create_app()`，绕过了 conftest 的 session 覆盖，于是连上了真实的 `dev.db`。**本地 141 个测试全绿，CI 上直接 `no such table`**。修完那一个测试只要十秒，真正的修法是在 conftest 里把 `DATABASE_URL` 提前指向内存库——**让同类疏忽以后当场大声失败，而不是静悄悄报绿**。

跑起来才暴露 **404 被 SPA 兜底吞掉了**

深链接兜底只看 `Accept: text/html`，于是浏览器访问一个不存在的 `/api/...` 会拿到 **200 + 首页 HTML**。fetch 不受影响（发的是 `*/*`），但**一个错误的 URL 渲染成看起来正常的页面，比 404 更糟**。把 `/api` 排除在兜底之外，并同时钉进单元测试和镜像冒烟测试。

环境问题 **两个非代码障碍**

其一：Docker Desktop 报 WSL 错误——`wsl.exe` 存在但那只是系统自带的壳，真正的 WSL 没装。`wsl --install --no-distribution` 加重启解决。其二：`docker-credential-desktop not found`——凭据助手不在 shell 的 PATH 里；我们拉的全是公开镜像，`auths` 本来就是空的，用一份不带 `credsStore` 的临时配置绕过。**这两个都跟代码无关，也都不会在 Render 上出现。**

另外还有一处属于"部署最容易翻车"的类型：Neon 给的连接串是 `postgresql://`，SQLAlchemy 会把它解析成 psycopg2，而项目装的是 psycopg3。**在配置层统一改写前缀**，这样连接串可以从面板原样粘贴——否则这一步会在 Render 上以一条完全看不懂的 `ModuleNotFoundError` 报出来。

07

清单

- [x] WSL 安装、Docker Desktop 可用
- [x] Neon 建库、4 条迁移执行、39 个语料包灌入（1609 句 / 0 未翻译 / 15327 词条）
- [x] 本地 `docker build` + `docker run` 连真实 Neon 跑通
- [x] CI 三个 job 全绿
- [ ] Render → Blueprints → 指向 `shaw4dev/vidlingo`，填 5 个环境变量
- [ ] 拿到公网 URL，验证一遍，写进 README 顶部
- [ ] **Neon 面板重置数据库密码**，只更新 Render 上那一个变量
- [ ] 本地 Vite 和后端重启——API 路由挪到 `/api` 了，跑着的进程是旧的

RENDER 面板要填的环境变量

```
DATABASE_URL          Neon 连接串，原样粘贴
ANTHROPIC_API_KEY     DeepSeek 的 key
ANTHROPIC_BASE_URL    https://api.deepseek.com/anthropic
TRANSLATE_MODEL       deepseek-v4-pro
```